

Cover Page

Project Title: Personal Financial Tracking Tool

Your Name: Chan-Chun Yen

Student ID: 923430102

GitHub Username: Jasonryentw

Milestone	Date Submitted
M1	09-18-2024
M2	10-01-2024
M3	10-16-2024

Table of Content

Table of Contents	
Cover Page	Page 1
Table of Contents	Page 2
Project Description	Page 3 - 4
Functional Database Requirements	Page 5 - 8
Non-functional Database Requirements	Page 9 - 10
Entity Relationship Diagram (ERD)	Page 11
Entity Set Description	Page 12 - 15
Enhanced Entity-Relationship (EER)	Page 16
Normalization Techniques Used	Page 17

Project Description

- Describe the motivations for creating this database system and the problems you aim to solve (high level only).

-

I usually keep track of my expenses in Excel spreadsheets, but when I need to review my spending, I have to use a summation tool to calculate the total expenses for a month, which is a bit inconvenient. I started thinking about whether there's a tool that can visualize all my expenses, allowing me to customize the date range and generate charts for a clearer view of my spending. Therefore, I decided to create a Personal Financial Tracking Tool that offers customizable visualizations.

- Provide a high-level, non-technical description of your database system.

-

My Personal Financial Tracking Tool is easy to use and allows users to record income, expenses, and savings, as well as compare their personal budget with actual spending. It also visualizes all the data through charts, enabling users to clearly understand their spending categories.

-

- Describe the unique features (high level only) you plan to implement in your database system based on your research. Include features that exist in other similar database systems that you will improve in your system.

-

As a student, I often compare my expenses from the previous month to those of the current month. However, I have noticed that many financial tracking tools lack a historical comparison feature. Even those that do have this feature often do not allow for customizable comparison dates. Therefore, I want to develop a simple yet highly flexible historical comparison feature.

- Name at least two existing software tools or products currently on the market that would benefit from using your database system. Explain in detail why they would benefit.

If YNAB and Daak had a historical comparison feature, it would allow their users to more easily observe financial comparisons for specific time periods. This feature would be particularly useful for students, who often have irregular income and expenses. It would make it easier for them to track their financial activities and could attract a broader range of users to these apps.

- Create at least three use cases that define the most important services your database will provide for different users and situations.

Case 1: Simplified Visualization

To ensure that the general public can easily understand the visualized charts, they will be presented using simple and intuitive formats such as pie charts or bar graphs. This approach will make the information accessible to users of all ages and educational backgrounds.

Case 2: Historical Comparing

Users who want to assess the growth of their finances or observe short-term changes in their income and expenses can use the historical comparison feature to customize and select specific time periods for reviewing financial changes.

Case 3: Budget Target

Many people often overspend, sometimes exceeding their income, which can lead to drawing from savings. Using the budget target feature allows users to plan their budget for specific items in advance, helping them clearly understand their remaining budget and prevent overspending.

Functional Database Requirements

1. User

- 1.1 A user shall create at most one account.
- 1.2 A user shall have a unique ID.
- 1.3 A user shall have a username and password.

2. Account

- 2.1 An account shall have one user.
- 2.2 An account shall have a unique ID.
- 2.3 An account shall have a creation date.
- 2.4 An account shall have a role type (admin or customer).

3. Payment Transaction

- 3.1 A Payment Transaction shall be linked to one RegisteredUser.
- 3.2 A Payment Transaction shall have a amount.
- 3.3 A Payment Transaction shall have a store name.
- 3.4 A Payment Transaction shall have a date.
- 3.5 A Payment Transaction shall have a Payment category.
- 3.6 A Payment Transaction shall have a note.

4. Payment Category

- 4.1 A Payment category shall have a unique ID.
- 4.2 A Payment category shall have a type name.
- 4.3 A Payment category shall be used by many Expense.

4.4 A Payment category shall have a color code.

5. Expense

5.1 A expense shall be linked to one RegisteredUser.

5.2 A expense shall have many Payment transactions.

5.3 A expense shall have a unique ID.

6. Earnings Category

6.1 A Earnings category shall have a unique ID.

6.2 A Earnings category shall have a type name.

6.3 A Earnings category shall be used by many Income.

6.4 A Earnings category shall have a color code.

7. Income transaction

7.1 A Income Transaction shall be linked to one RegisteredUser.

7.2 A Income Transaction shall have an amount.

7.3 A Income Transaction shall have a source name.

7.4 A Income Transaction shall have a date.

7.5 A Income Transaction shall have an Earning category.

7.6 A Income Transaction shall have a note.

8. Income

8.1 An Income shall be linked to one RegisteredUser.

8.2 An Income shall have many Income transaction.

8.3 An income shall have a unique ID.

9. Saving

9.1 A saving shall be linked to one RegisteredUser.

9.2 A saving shall have an amount.

9.3 A saving shall have a target goal note.

10. Budget

10.1 A budget shall be linked to one RegisteredUser.

10.2 A budget shall have one payment category.

10.3 A budget shall have one amount.

11. Chart

11.1 A Chart shall have one expense.

11.2 A Chart shall have one Income.

11.3 A Chart shall have one saving.

11.4 A Chart shall have one type.

12. Visualize

12.1 A Visualize shall have many Charts.

12.2 A Visualize shall be linked to one Dashboard.

12.3 A Visualize shall have one Visualize preference

13. Visualize Preference

13.1 A Visualize Preference shall have a start date.

13.2 A Visualize Preference shall have an end date.

13.3 A Visualize Preference shall be linked to one Visualize.

14. Historical Comparing

14.1 A Historical comparing shall have many Charts.

14.2 A Historical comparing shall have one Expense.

14.3 A Historical comparing shall have one Income

14.4 A Historical comparing shall have one saving

14.5 A Historical comparing shall have one Historical comparing preference.

15. Historical Comparing preference

15.1 A Historical Comparing preference shall have a start date.

15.2 A Historical Comparing preference shall have an end date.

15.3 A Historical Comparing preference shall be linked to one Historical comparing.

16. Budget Comapring

16.1 A Budget Comapring shall have many Charts.

16.2 A Budget Comapring shall have one Expense.

- 16.3 A Budget Comparing shall have one budget
- 16.4 A Budget Comparing shall have one Budget comparing preference.

17. Budget Comparing preference

- 17.1 A Historical Comparing preference shall have a start date.
- 17.2 A Historical Comparing preference shall have an end date.
- 17.3 A Historical Comparing preference shall be linked to one Historical comparing.

18. Dashboard

- 18.1 A Dashboard shall have one visualize.
- 18.2 A Dashboard shall have one historical comparing.
- 18.3 A Dashboard shall have one budget comparing.
- 18.4 A Dashboard shall be linked to one RegisteredUser.

19. Notification

- 19.1 A notification shall have an alert message.
- 19.2 A notification shall be linked to one RegisteredUser.
- 19.3 A notification shall have an account id.

20. Report

- 20.1 A report shall be linked to one RegisteredUser.
- 20.2 A report shall have one unique id.
- 20.3 A report shall have a date.

21. RegisteredUser

- 21.1 A RegisteredUser shall have one account.
- 21.2 A RegisteredUser shall have one expense, one income and one saving.
- 21.3 A RegisteredUser shall have one notification.
- 21.4 A RegisteredUser shall have many report.
- 21.5 A RegisteredUser shall have one dashboard.

Non-functional Database Requirements

1. Performance

- 1.1 The database system shall support concurrent transactions.
- 1.2 The database system shall process inquiries in a few seconds.
- 1.3 The database system shall optimize time when accessing Expense and Income.

2. Security

- 2.1 2FA will be implemented for user login.
- 2.2 All financial data shall be encrypted while in transaction.
- 2.3 The database system shall automatically backup every 24 hours.

3. Scalability

- 3.1 The database system shall support 1000 users and their financial records.
- 3.2 The database system shall support dynamic allocation for the increased using charts.
- 3.3 The database system shall partial the large-size data into small pieces.

4. Capability

- 4.1 The database system shall support the export of transaction records in multiple formats.
- 4.2 The visualization shall allow the user to generate their customized graph.
- 4.3 The report shall allow the user to choose items that they want to display.

5. Environmental

- 5.1 The system shall use less RAM for operations.
- 5.2 The system shall be run on the Linux platform.
- 5.3 The system shall consume less power energy for operations.

6. Coding Standard

- 6.1 Variables shall be named clearly and understandable.

6.2 Each line shall be clearly stated by comments to make it readable.

6.3 Codes shall be updated to Github regularly to keep tracking.

7. Storage

7.1 The database system shall support 1TB for financial records.

7.2 The database system shall compress the large-size data without affecting retrieving performance.

7.3 The database system shall archive the data that hasn't been used for 5 years.

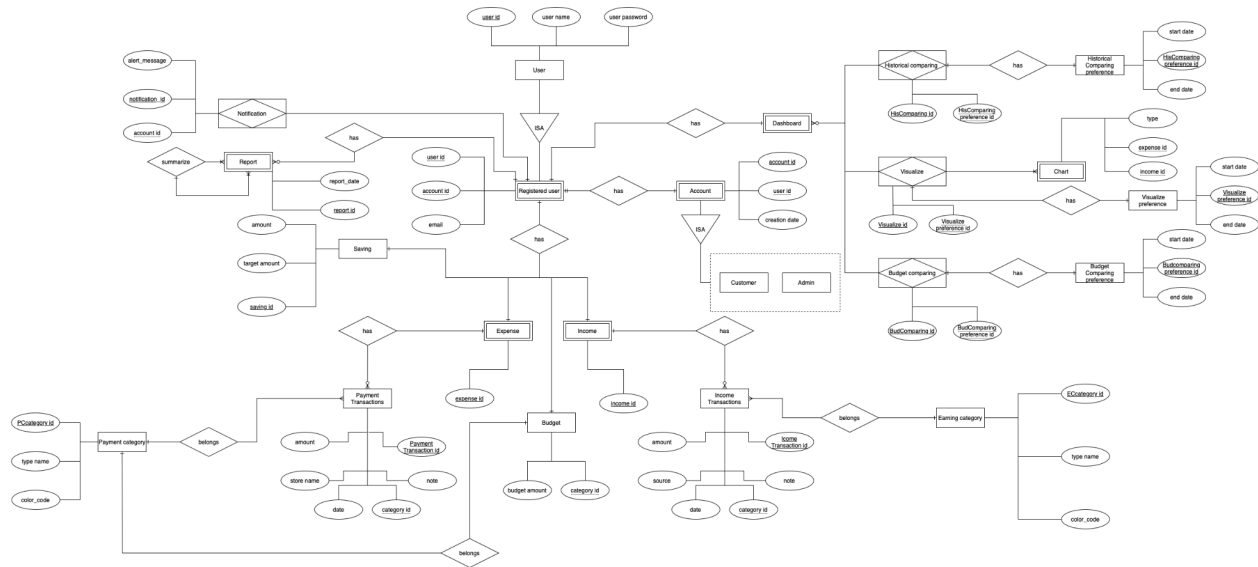
8. Privacy

8.1 The database shall be encrypted and only available for authorized admin.

8.2 The database system shall let the user choose how long they want to keep their data.

8.3 The database system shall ask for agreement about collecting data for analysis.

Entity Relationship Diagram (ERD)



Entity Set Description

1. User (Strong)

- user_id: numeric, primary key
- user_name: alphanumeric
- user_password: alphanumeric

2. RegisteredUser (Weak)

- user_id: numeric, foreign key
- email: alphanumeric
- account_id: numeric, foreign key

3. Account (Weak)

- account_id: numeric, primary key
- creation_date: date-time
- expense_id: numeric, primary key and foreign key (references Expense entity set)
- income_id: numeric, primary key and foreign key (references Income entity set)
- budget_id: numeric, primary key and foreign key (references budget entity set)
- saving_id: numeric, primary key and foreign key (references saving entity set)

4. PaymentTransaction (Strong)

- Payment_transaction_id: numeric, primary key
- account_id: numeric, primary key and foreign key (references Account entity set)
- amount: numeric
- store_name: alphanumeric
- date: date-time
- PCcategory_id: numeric, foreign key (references PaymentCategory entity set)
- note: alphanumeric

5. PaymentCategory (Strong)

- PCcategory_id: numeric, primary key
- type_name: alphanumeric
- color_code: alphanumeric

6. Expense (Weak)

- expense_id: numeric, primary key
- account_id: numeric, primary key and foreign key (references Account entity set)

- payment_transactions: numeric, foreign key (references PaymentTransaction entity set)

7. EarningCategory (Strong)

- ECcategory_id: numeric, primary key
- type_name: alphanumeric
- color_code: alphanumeric

8. IncomeTransaction (Weak)

- Income_transaction_id: numeric, primary key
- account_id: numeric, primary key and foreign key (references Account entity set)
- amount: numeric
- source_name: alphanumeric
- date: date-time
- ECcategory_id: numeric, foreign key (references EarningsCategory entity set)
- note: alphanumeric

9. Income (Weak)

- income_id: numeric, primary key
- account_id: numeric, primary key and foreign key (references Account entity set)
- income_transactions: numeric, primary key and foreign key (references

IncomeTransaction entity set)

10. Saving (Weak)

- saving_id: numeric, primary key
- account_id: numeric, primary key and foreign key (references Account entity set)
- amount: numeric
- target_goal_note: alphanumeric

11. Budget (Weak)

- budget_id: numeric, primary key
- account_id: numeric, primary key and foreign key (references Account entity set)
- PCcategory_id: numeric, foreign key (references PaymentCategory entity set)
- amount: numeric

12. Chart (Weak)

- chart_id: numeric, primary key
- expense_id: numeric, primary key and foreign key (references Expense entity set)
- income_id: numeric, primary key and foreign key (references Income entity set)
- saving_id: numeric, primary key and foreign key (references Saving entity set)

- type: alphanumeric

13. Visualize (Associative)

- visualize_id: numeric, primary key
- account_id: numeric, composite primary key and foreign key (references Account entity set)
- visualize_preference_id: numeric, composite foreign key (references VisualizePreference entity set)

14. VisualizePreference (Strong)

- visualize_preference_id: numeric, primary key
- start_date: date-time
- end_date: date-time

15. HistoricalComparing (Associative)

- HisComparing_id: numeric, primary key
- expense_id: numeric, composite primary key and foreign key (references Expense entity set)
- income_id: numeric, composite primary key and foreign key (references Income entity set)
- saving_id: numeric, composite foreign key (references Saving entity set)
- HisComparing_preference_id: numeric, composite foreign key (references HistoricalComparingPreference entity set)

16. HistoricalComparingPreference (Strong)

- HisComparing_preference_id: numeric, primary key
- start_date: date-time
- end_date: date-time

17. BudgetComparing (Associative)

- BudComparing_id: numeric, primary key
- expense_id: numeric, composite primary key and foreign key (references Expense entity set)
- budget_id: numeric, composite foreign key (references Budget entity set)
- Budcomparing_preference_id: numeric, composite foreign key (references BudgetComparingPreference entity set)

18. BudgetComparingPreference (Strong)

- Budcomparing_preference_id: numeric, primary key

- start_date: date-time
- end_date: date-time

19. Dashboard (Weak)

- dashboard_id: numeric, primary key
- visualize_id: numeric, composite primary key and foreign key (references Visualize entity set)
- HisComparing_id: numeric, composite primary key and foreign key (references HistoricalComparing entity set)
- BudComparing_id: numeric, composite primary key and foreign key (references BudgetComparing entity set)
- account_id: numeric, composite primary key and foreign key (references Account entity set)

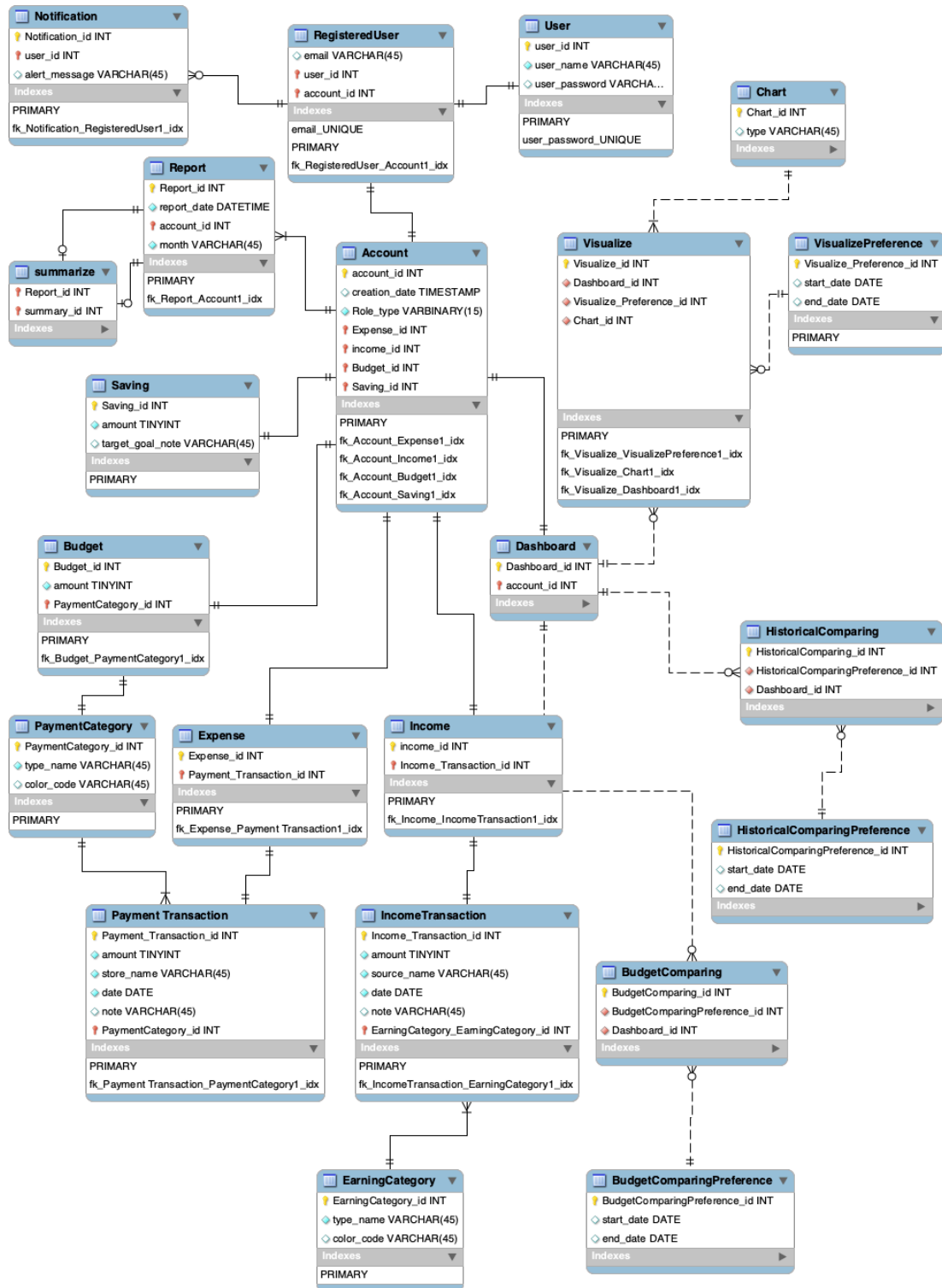
20. Notification (Associative)

- notification_id: numeric, primary key
- alert_message: alphanumeric
- account_id: numeric, composite primary key and foreign key (references Account entity set)

21. Report (Weak)

- report_id: numeric, primary key
- account_id: numeric, composite primary key and foreign key (references Account entity set)
- report_date: date-time

Enhanced Entity-Relationship (EER)



Normalization Techniques Used

First Normal Form (1NF) Atomic

The attributes in each table are atomic. There's no repetitive or multi-value attribute inside the table. Thus, all the entities are already in 1NF.

Second Normal Form (2NF) partial dependency

The tables are in 1NF

Account:

The PK/FK account_id should be unique here, and the other columns (expense_id, income_id, etc.) depends on it. So it's in 2NF.

Expense and Income:

Both tables own their primary key (expense_id and income_id) and the other fields depends on them, so they are in 2NF.

Associative Table (Visualize, Historical comparing and Budget comparing):

These tables use composite keys (Dashboard_id) as primary key and the fields depends on it, so they are in 2NF.

Third Normal Form (3NF) transitive dependencies

The tables are in 2NF

User:

User_id is the primary key and there's no transitive dependencies between non-key attributes(user_name, user_password).

RegisteredUser:

This table has foreign key user_id and all the other attributes (email, account_id) depends on it, so there's no transitive dependency.